International Journal of Electrical and Computer Engineering (IJECE)
Vol. 16, No. 5, October 2026, pp. 2664∼2671
ISSN: 2088-8708, DOI: 10.11591/ijece.v16i5.pp2664-2671

Auditable patch validity for lightweight language-model software repair

Rishabh Singh
Independent Researcher, Seattle, Washington, United States

Article Info

Article history:

Received Jun 11, 2026
Revised Aug 12, 2026
Accepted Aug 19, 2026

Keywords:

Artifact provenance
Automated program repair
Code generation
Repository validation
Software quality
SWE-bench Lite

ABSTRACT

Terminal resolve rate obscured where repository-level language-model patches lost utility between generation and execution. This study addressed that measurement gap by proposing an auditable patch-validity framework whose conceptual novelty was to represent each artifact as monotonic survival through six evidence-linked checkpoints: non-empty generation, syntactic validity, repository applicability, harness completion, behavioral resolution, and review readiness. The first five were measured from archived generations, normalization records, and Docker-harness outcomes; review readiness remained a proposed deployment check. In a 300-artifact diagnostic slice, 64 failed syntax, 77 failed applicability, 151 reached execution but remained unresolved, and 8 resolved. Across 1,600 larger-run artifacts, 776 (48.5%) were invalid or inapplicable and 22 resolved. Staged prompting improved completion in some configurations, but its strongest diagnostic resolution advantage, 4/50 versus 1/50, was exploratory ($p = 0.25$) and reversed at larger scale. By converting terminal outcomes into syntactic, repository-state, and semantic failure tiers, the framework enabled low-cost pre-execution filtering, targeted remediation, and auditable continuous integration (CI) resource allocation. It was a measurement and triage instrument, not a new repair algorithm.

This is an open access article under the CC BY-SA license.

Corresponding Author:

Rishabh Singh
Independent Researcher
Seattle, Washington, United States
Email: rishabhsingh@gwu.edu

1. INTRODUCTION

Executable function-level benchmarks established test-based evaluation for language-model code generation [1]-[4]. Repository-level work then added codebase context, issue descriptions, multi-file changes, and regression tests through RepoBench and SWE-bench [5], [6]. These settings exposed a practical gap between plausible generated text and a usable repository patch. A response can be non-empty yet contain a malformed diff, edit a nonexistent path, depend on stale hunk context, reverse a change already present at the benchmark commit, or apply cleanly without correcting behavior.

Recent systems improved repository localization, tool interaction, and iterative repair. SWE-agent studied agent-computer interfaces, Agentless separated localization and repair, OpenHands provided a general software-agent platform, and RepairAgent combined autonomous planning with repair tools [7]-[10]. Repository-aware context extraction, retrieval-augmented repair, and defect-aware staging also improved patch generation [11]-[13]. Live evaluation, synthetic training tasks, feature implementation, and security-sensitive completion broadened repository-level assessment [14]-[17]. Work on compact code models [18], [19], evidence-

grounded bug detection [20], software-lifecycle use [21], and prompt-orchestrated testing [22] likewise emphasized reliability. However, terminal resolve rate still compressed structurally different failures into one number.

This paper addressed that measurement gap. Its conceptual novelty was a monotonic evidence model that linked each patch to generation, normalization, application at the exact checkout, harness execution, behavioral resolution, and prospective review. Failure was therefore attributed at the earliest unsupported transition. The resulting instrumentation layer was not a competing repair algorithm. It separated syntactic failures that could be rejected before checkout, repository-state failures that required path or hunk alignment, and semantic failures that reached execution but did not resolve the issue. Five checkpoints were observable from the archived experiment. Repository applicability and harness completion remained conceptually distinct although they coincided numerically in the available summaries; review readiness was explicitly proposed rather than inferred. Operationally, the framework served as computing-systems evaluation infrastructure for engineering pipelines that deploy language models to generate, validate, and route repository changes.

The distinction from prior evaluation was operational. Code-generation benchmarks tested final functional behavior, and repair agents optimized how a patch was produced. The proposed framework treated artifact lineage as the object of measurement. It could be applied after inference to different agents without changing their search procedure, and it exposed losses that a terminal success count could not attribute.

The framework was instantiated on SWE-bench Lite using direct and staged prompting with OpenAI gpt-4o-mini, Qwen2.5-Coder-7B-Instruct, and Qwen2.5-Coder-32B-Instruct-AWQ. A fixed 50-instance slice supported artifact inspection, and larger runs tested whether diagnostic patterns persisted. Across 1,600 larger-run artifacts, 776 (48.5%) were invalid or failed applicability before behavioral resolution, and only 22 resolved. The resulting evidence made three novel contributions: i) a formal, auditable checkpoint model; ii) paired evidence showing that improved executability did not necessarily imply improved resolution; and iii) a reproducible three-tier triage taxonomy tied to specific engineering responses. As a common audit contract, these contributions could support cross-system comparison, intercept cheaply detectable failures before expensive continuous integration (CI), and route surviving artifacts to the appropriate engineering intervention.

2. PATCH-VALIDITY FRAMEWORK

For artifact i, let $s_{i,k} \in \{0,1\}$ indicate survival through checkpoint k. The ordered construction required

$$s_{i,k+1} \leq s_{i,k}, \qquad k = 1, \ldots, 5, \tag{1}$$

so an artifact could not pass a later checkpoint after failing an earlier one. Checkpoint attrition was

$$A_k = \sum_i (s_{i,k} - s_{i,k+1}). \tag{2}$$

This simple invariant prevented later outcomes from concealing earlier failures and made all reported counts mechanically auditable.

Table 1 defines the six checkpoints. Non-empty generation and diff syntax were available from raw and normalized outputs. Repository applicability required target-path alignment and a successful patch application against the exact base commit. Harness completion recorded whether tests executed, while behavioral resolution was the official SWE-bench result. Review readiness covered maintainability, regression, and security assessment; it was not automatically assigned. ISO/IEC 25010:2023 supplied a supporting quality interpretation, not the framework itself [23].

Table 1. Ordered checkpoints, evidence, and primary response

Checkpoint	Recorded evidence	Primary response to failure
Non-empty generation	Raw output contains a candidate patch	Constrain output or retry
Syntactic validity	Unified diff parses or is transparently repairable	Validate before checkout
Repository applicability	Paths exist and patch applies at the base commit	Refresh context or rebase hunks
Harness completion	Evaluator reaches task-specific test execution	Diagnose environment or execution
Behavioral resolution	Official tests mark the issue fixed	Iterate on behavior and tests
Review readiness	Human review, regression, maintainability, and security checks	Revise, approve, or reject

The ordering did not assume statistical independence; adjacent survival indicators were dependent by construction. Empirical separability instead meant observing the conditional transition between two gates.

A patch that passed `git apply --check` but then failed container startup, dependency installation, test discovery or invocation, or timed out before task-specific execution would have $s_{i,3} = 1$ and $s_{i,4} = 0$ and would contribute to A_3. It would be classified as harness attrition, not repository-state misalignment or semantic insufficiency, because no behavioral result was produced. The present summaries contained no observed $s_{i,3} = 1, s_{i,4} = 0$ cases, so $A_3 = 0$ in this dataset; this equality did not make the checkpoints conceptually equivalent. Future runs should preserve separate application, environment-startup, test-invocation, and completion statuses.

3. METHOD

3.1. Research questions and experimental design

The study asked:

RQ1: How much repair utility was lost at each checkpoint?
RQ2: Which syntactic, repository-state, and semantic failures explained the losses?
RQ3: Whether direct-versus-staged patterns from the diagnostic slice persisted in larger runs?

Each SWE-bench Lite instance began at the benchmark base commit. A deterministic context builder ranked text files by overlap between problem-statement tokens and path or content tokens, retained at most 18 files, truncated each file at 12,000 characters, and capped combined context at 90,000 characters. Direct mode issued one patch-generation request. Staged mode first requested one to five candidate paths in strict JSON and then generated a patch with those files prioritized. Both modes required unified `diff --git` output.

The diagnostic study used the same fixed 50 tasks for all six backend-mode configurations. The extension evaluated all 300 Lite tasks for gpt-4o-mini and Qwen 32B AWQ, and a deterministic 200-task lexicographic subset for Qwen 7B. Thus, the larger study contained 1,600 artifacts: two 300-task modes for each of two backends and two 200-task modes for Qwen 7B. The Qwen models ran without fine-tuning; the 32B deployment used 4-bit AWQ quantization. Earlier 50-task predictions remained immutable seed data and extension scripts skipped existing instance identifiers.

3.2. Artifact pipeline and analysis

Four representations were archived separately: literal raw generation, normalized SWE-bench JSONL, a repair-normalized diff when transparent correction was possible, and the artifact evaluated by the Docker harness. Missing diff headers and irreparable outputs remained failures rather than disappearing from denominators. Sidecar records included instance, backend, mode, token counts, elapsed time, non-emptiness, header status, and recoverable errors.

The replication package also contained an executable command-line auditor. For each prediction it recorded non-emptiness, parsed diff structure, target paths, optional path existence, optional applicability against a supplied checkout, harness completion, resolution, and a conservative failure tier. A fixture suite covered empty output, malformed diff, wrong path, stale hunk, applicable-but-unresolved, and resolved cases. The auditor never inferred review readiness, and a replay script reproduced the 1,600-artifact headline from frozen summaries.

Mechanical outcomes came directly from parser, application, and harness logs. Fine-grained application failures were descriptively grouped as malformed or truncated diff, stale hunk, wrong or missing path, and reversed or already-applied edit. These labels were then mapped to the three failure tiers in Table 2. Because one author performed the primary descriptive grouping, a blinded 60-item sensitivity check used two separately executed automated coding agents and the fixed coding guide. Both agents could share model or guide-induced biases; their agreement measured coding-guide repeatability under those agents, not category truth or independent human inter-rater validity.

Table 2. Artifact-grounded diagnostic tiers aggregated over six configurations

Outcome tier	Included primary labels	Count	Percent	Triage
Syntactic invalidity	Malformed or truncated diff	64	21.3%	Parse, repair transparently, or regenerate
Repository-state misalignment	Stale hunk, wrong path, reversed edit	77	25.7%	Refresh checkout context and rebase
Semantic insufficiency	Applies but remains unresolved	151	50.3%	Run targeted tests and iterate
Behaviorally resolved	Official evaluator reports resolved	8	2.7%	Continue to review-readiness checks

Paired direct-versus-staged resolution outcomes were analyzed with the exact McNemar test because each mode used the same tasks [24]. All tests were two-sided and exploratory. Public SWE-bench summaries

were re-ingested only to check top-level vocabulary compatibility, not as protocol-matched patch-level replications.

4. RESULTS AND DISCUSSION

4.1. Diagnostic checkpoint attrition

Table 3 and Figure 1 answer RQ1 for the 50-task slice. Every configuration returned 50 non-empty artifacts, but survival diverged at syntax and applicability. Staging increased completion from 23 to 29 for gpt-4o-mini and from 29 to 33 for Qwen 7B. For Qwen 32B AWQ, it changed completion only from 22 to 23. The strongest terminal difference was 1/50 direct versus 4/50 staged for Qwen 32B AWQ. Its paired table contained three staged-only successes and no direct-only success, but the exact result remained exploratory ($p = 0.25$).

Table 3. Cumulative diagnostic-slice checkpoint counts (50 tasks per row)

Backend	Mode	Non-empty	Syntax-valid	Completed	Resolved
gpt-4o-mini	Direct	50	31	23	0
gpt-4o-mini	Staged	50	37	29	1
Qwen 7B	Direct	50	42	29	1
Qwen 7B	Staged	50	42	33	1
Qwen 32B AWQ	Direct	50	44	22	1
Qwen 32B AWQ	Staged	50	40	23	4

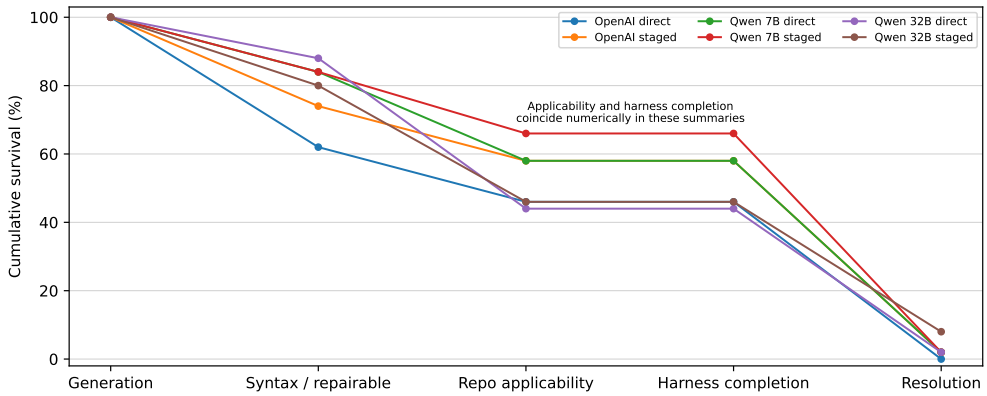


Figure 1. Cumulative survival through the measured checkpoints in the 50-instance diagnostic slice. Applicability and harness completion coincided in the available summaries; review readiness was proposed and therefore not plotted

Aggregating the 300 diagnostic artifacts clarified the engineering interpretation as shown in Table 2. Sixty-four artifacts (21.3%) failed syntactically and 77 (25.7%) survived syntax but failed repository applicability. Of the 159 that reached execution, 151 remained unresolved. These tiers required different interventions: parser checks, checkout-aware context and rebasing, or behavioral iteration. The decomposition therefore provided information unavailable from the eight terminal successes alone.

Artifact inspection illustrated why the boundary mattered. Some outputs ended mid-hunk or combined prose with an incomplete diff and could be rejected without cloning a repository. Other patches were structurally valid but targeted absent paths, searched for stale context, or reversed changes already present at the pinned commit; additional prompting without refreshed repository state would not address those failures. A third group applied and executed yet changed only part of the required behavior or missed regressions. These locally plausible patches consumed the most validation effort because syntax and applicability alone could not distinguish them from successful repairs.

4.2. Larger evaluation and prompting-mode stability

Table 4 answers RQ3. The apparent Qwen 32B staged advantage did not persist: the 300-task result was 8 direct versus 6 staged resolutions. gpt-4o-mini resolved 3 direct versus 1 staged, while Qwen 7B resolved 2 in each mode. Exact paired p values were 0.625, 0.727, and 1.000, respectively. Staging nevertheless raised

Qwen 7B completion from 91 to 114. This separation between executability and resolution was precisely the behavior the checkpoint framework was designed to expose.

Table 4. Larger SWE-bench Lite evaluation

Backend	Mode	Tasks	Completed	Invalid/apply	Unresolved	Resolved
gpt-4o-mini	Direct	300	151	149	148	3
gpt-4o-mini	Staged	300	157	143	156	1
Qwen 32B AWQ	Direct	300	155	145	147	8
Qwen 32B AWQ	Staged	300	156	144	150	6
Qwen 7B	Direct	200	91	109	89	2
Qwen 7B	Staged	200	114	86	112	2

Across all larger rows, 776 of 1,600 artifacts (48.5%) were invalid or failed applicability, with per-configuration rates from 43.0% to 54.5%. Another 802 applied but remained unresolved, and 22 resolved. The Qwen 32B reversal required caution because the earlier 50 tasks were immutable seeds: that subset favored staging 4 to 1, whereas the 250 extension-only tasks favored direct mode 7 to 2. The Qwen 7B extension was also a deterministic subset rather than a random sample. These design facts prevented a causal claim that one prompt mode was superior.

The model-assisted recoding sensitivity check produced 93.3% primary-label agreement (Cohen's $\kappa = 0.90$) and 93.3% tier agreement ($\kappa = 0.89$). All four disagreements concerned the boundary between malformed or truncated diffs and stale hunks, and all high-confidence items agreed. These values supported repeatability under the fixed guide and selected coding agents only; they neither validated the taxonomy independently nor replaced human agreement assessment.

4.3. Operational implications

The evidence answered RQ2 by localizing three intervention points. Syntactic failures could be filtered before repository checkout. Repository-state failures called for exact-commit path checks, refreshed retrieval, or hunk rebasing. Applied-but-unresolved patches required test-guided iteration rather than formatting repair. A deployment pipeline should therefore archive raw and normalized artifacts, validate diff structure, run `git apply --check` against the pinned commit, record harness completion separately, and reserve human review for artifacts that survive behavioral validation.

This ordering also supported resource allocation. Parsing and applicability checks were inexpensive and deterministic, while container execution and human review were comparatively costly. Reporting survival counts therefore described both research performance and likely operational load. A system with unchanged resolve rate but fewer malformed or stale patches could still reduce wasted CI capacity; a system that applied reliably but failed behaviorally needed better testing or repair reasoning instead of another format normalizer.

Public RAG + Claude 2 and SWE-agent + GPT-4 summaries could be represented using the same top-level vocabulary [25], [26]. This established reporting compatibility only; public summaries lacked the protocol-matched artifact evidence needed to estimate every intermediate checkpoint. The framework was consequently a companion to resolve rate, not a replacement for it. Its value was greatest when terminal successes were sparse: systems with the same solve count could impose very different parser, checkout, and CI costs.

5. THREATS TO VALIDITY

5.1. Internal validity

Fixed prompts and a deliberately lightweight retriever limited performance but made attrition reproducible. Earlier predictions were retained unchanged, and all empty or invalid artifacts remained in denominators. Fine-grained labels were single-author descriptive coding. Automated recoding tested repeatability under a shared guide, could inherit common model biases, and was not independent human validation.

5.2. Construct validity

Harness completion was not successful repair, and benchmark resolution was not deployment readiness. Separating raw, normalized, repair-normalized, and evaluated artifacts made preprocessing visible. Applicability and completion coincided in the archived summaries. Their observed transition attrition was therefore zero, but the data could not characterize applicable patches that failed during environment or test execution.

5.3. External validity

Results covered SWE-bench Lite, three accessible backend configurations, and two prompt modes. Repository-state misalignment rates were conditional on the fixed 18-file, 12,000-character-per-file, and 90,000-character context limits. The 32B result characterized an AWQ-quantized deployment, and Qwen 7B used a 200-task deterministic subset. Stronger models, richer retrieval, other languages, security review, and human-in-the-loop workflows may change the distribution.

5.4. Conclusion validity

Resolved cases and discordant pairs were sparse. Exact McNemar results were descriptive and underpowered, not evidence of equivalence. The study's supported claim was checkpoint-level localization of failure, not prompting-mode superiority.

6. CONCLUSION

This study treated repository-level language-model repair as an auditable attrition process. The proposed framework separated non-empty generation, syntax, repository applicability, harness completion, behavioral resolution, and review readiness. Applied to SWE-bench Lite, it showed that 776 of 1,600 larger-run artifacts failed before behavioral evaluation and that most applicable patches still remained unresolved. Staged prompting sometimes improved executability, but those gains did not reliably become resolved issues at scale. The artifact-grounded failures reduced to three actionable tiers: syntactic invalidity, repository-state misalignment, and semantic insufficiency. Reporting these checkpoints with resolve rate made negative results diagnostically useful and supplied a conservative triage procedure for CI and review. With separate application and execution logging, the framework could improve cross-system comparability and reduce avoidable validation cost even before terminal solve rates changed. Future work should use independent human coders, evaluate stronger agents and additional languages, and validate review readiness with maintainability, security, and regression assessment.

ACKNOWLEDGMENTS

OpenAI ChatGPT (GPT-5, for the current manuscript revision), OpenAI Codex (GPT-5-based), and Anthropic Claude Code (service version available in June 2026) were used for language editing, organization, LaTeX and code-support tasks, and the disclosed automated recoding sensitivity analysis. They did not receive authorship credit. The author reviewed the outputs and verified the manuscript, calculations, citations, and conclusions.

FUNDING INFORMATION

This research received no external funding and was not supported by any research grant or contract. The author states that no funding was involved.

AUTHOR CONTRIBUTIONS STATEMENT

This journal uses the Contributor Roles Taxonomy (CRediT) to recognize individual author contributions, reduce authorship disputes, and facilitate collaboration.

Name of Author	C	M	So	Va	Fo	I	R	D	O	E	Vi	Su	P	Fu
Rishabh Singh	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓			✓	

C : **C**onceptualization
M : **M**ethodology
So : **So**ftware
Va : **Va**lidation
Fo : **Fo**rmal Analysis
I : **I**nvestigation
R : **R**esources
D : **D**ata Curation
O : Writing - **O**riginal Draft
E : Writing - Review & **E**diting
Vi : **Vi**sualization
Su : **Su**pervision
P : **P**roject Administration
Fu : **Fu**nding Acquisition

CONFLICT OF INTEREST STATEMENT

The author declares no known competing financial interests, personal relationships, or other conflicts of interest that could have appeared to influence the work reported in this paper.

DATA AVAILABILITY

The data that support the findings of this study, including scripts, prompts, predictions, evaluation summaries, and audit artifacts, are openly available in Zenodo at doi:10.5281/zenodo.20321563, record 20321563. The associated source code is available in the public GitHub repository. The archive also preserves earlier manuscript snapshots and a separate function-level pilot; neither is analyzed in the present study.

REFERENCES

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] Y. Lai *et al.*, "DS-1000: A natural and reliable benchmark for data science code generation," in *ICML'23: Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 18319–18345, doi: 10.5555/3618408.3619164.

[4] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *NIPS '23: Proceedings of the 37th International Conference on Neural Information Processing Systems*, vol. 36, 2023, doi: doi/10.5555/3666122.3667065

[5] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking repository-level code auto-completion systems," in *International Conference on Learning Representations*, pp. 1-19, 2024.

[6] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations*, 2024, pp. 54107-54157.

[7] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50528-50652.

[8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," in *Proceedings of the ACM on Software Engineering*, vol. 2, 2025, pp. 801-824, doi: 10.1145/3715754.

[9] X. Wang *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *The Thirteenth International Conference on Learning Representations*, 2025.

[10] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *ICSE '25: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[11] Y. Chen *et al.*, "When large language models confront repository-level automatic program repair: How well they done?" *arXiv preprint arXiv:2403.00448*, 2024.

[12] Z. Liu, X. Du, and H. Liu, "ReAPR: Automatic program repair via retrieval-augmented large language models," *Software Quality Journal*, vol. 33, 2025, doi: 10.1007/s11219-025-09728-1.

[13] R. Zhang, S. Zhang, and L. Xie, "Towards defect-type-aware adaptive program repair: A stage-wise approach with large language models," *Software Quality Journal*, vol. 34, 2026, doi: 10.1007/s11219-026-09751-w.

[14] L. Zhang *et al.*, "SWE-bench goes live," *arXiv preprint arXiv:2505.23419*, 2025.

[15] J. Yang *et al.*, "SWE-smith: Scaling data for software engineering agents," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, doi: 10.52202/085713-3239.

[16] W. Li *et al.*, "FEA-Bench: A benchmark for evaluating repository-level code generation for feature implementation," *arXiv preprint arXiv:2503.06680*, 2025.

[17] C. Shen *et al.*, "SecRepoBench: Benchmarking code agents for secure code completion in real-world repositories," *arXiv preprint arXiv:2504.21205*, 2025.

[18] V.-V. Nguyen, T.-V. Nguyen, H.-K. Nguyen, and D.-Q. Vu, "Parameter-efficient fine-tuning of small language models for code generation: A comparative study of Gemma, Qwen 2.5 and Llama 3.2," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 16, no. 1, pp. 278–287, 2026, doi: 10.11591/ijece.v16i1.pp278-287.

[19] B. Hui *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[20] C. Wang, W. Zhang, Z. Su, X. Xu, and X. Zhang, "Sanitizing large language models in bug detection with data-flow," in *Findings of the Association for Computational Linguistics: EMNLP 2024, 2024*, 2024, pp. 3790–3805, doi: 10.18653/v1/2024.findings-emnlp.217.

[21] J. S. Bhalla and M. K. Jodhka, "Leveraging large language models in the software development lifecycle: Opportunities and challenges," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 11, 2025, doi: 10.14569/IJACSA.2025.0161105.

[22] A. A. F. Antonio, A. Barrientos Padilla, and R. G. Siancas Garay, "Engineering prompt-orchestrated LLM workflows for automated test case generation in agile environments," *International Journal of Advanced Computer Science & Applications*, vol. 16, no. 12, 2025, doi: 10.14569/IJACSA.2025.0161272.

[23] ISO/IEC, *ISO/IEC 25010:2023 systems and software engineering–systems and software quality requirements and evaluation (SQuaRE)–product quality model*. Geneva, Switzerland: ISO, 2023.

[24] M. W. Fagerland, S. Lydersen, and P. Laake, "The McNemar test for binary matched-pairs data: Mid-p and asymptotic are better than exact conditional," *BMC medical research methodology*, vol. 13, art. 91, 2013, doi: 10.1186/1471-2288-13-91.

[25] SWE-bench, "Experiments: Open-sourced predictions, execution logs, trajectories, and results from model inference and evaluation runs," GitHub repository, accessed Jun. 2026. [Online]. Available: `https://github.com/SWE-bench/experiments`.

[26] OpenAI, "Introducing SWE-bench verified," 2024. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/.

BIOGRAPHY OF THE AUTHOR



Rishabh Singh received the Master of Science degree in computer science, with a specialization in data analytics, from George Washington University, Washington, DC, United States, in 2024, and the Bachelor of Engineering degree in computer science from Visvesvaraya Technological University, Karnataka, India, in 2020. He works in software engineering and conducts independent research on large language models for software engineering, automated program repair, repository-level evaluation, and reproducible software-quality measurement. For this study, he designed and implemented the experimental pipeline and auditor, curated and analyzed the artifacts, and prepared the manuscript and replication package. He can be contacted at rishabhsingh@gwu.edu.